Справочный материал

Класс и структура: общая картина

И класс, и структура — это способ создать **свой тип данных**: описать, из каких данных он состоит (поля) и что умеет (методы, свойства, конструкторы). Синтаксически они почти одинаковы:

```
public struct Pixel        // структура
{
    public byte R, G, B;
}

public class PixelClass    // класс
{
    public byte R, G, B;
}
```

Разница не в том, **что** можно написать внутри, а в том, **как переменные этих типов хранят данные**:

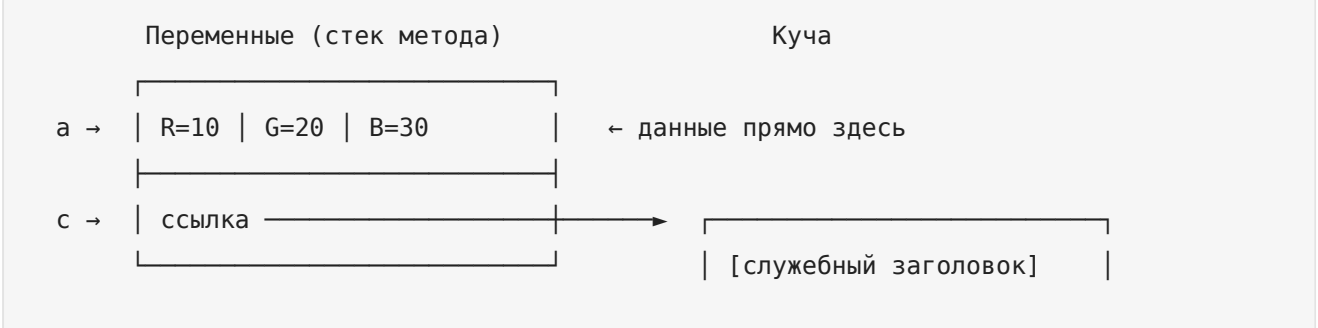
	`struct` (структура)	`class` (класс)
Категория типа	**значимый** (value type)	**ссылочный** (reference type)
Что лежит в переменной	сами данные (все поля)	ссылка (адрес) на объект в куче
`b = a;`	копируются **все поля** — получаем независимую копию	копируется **ссылка** — обе переменные смотрят на один объект
Передача в метод (без `ref`)	метод получает **копию данных**	метод получает **копию ссылки** на тот же объект
Значение по умолчанию	все поля = 0 / `false` / `null` (`default`)	`null` — объекта нет
Может быть `null`?	нет	да
`==` «из коробки»	не определён (ошибка компиляции)	сравнивает **ссылки** (тот же ли это объект)
`Equals` «из коробки»	сравнивает **поля**	сравнивает **ссылки**
Где живут данные	там, где объявлена переменная/поле	всегда в куче
Типичные примеры в .NET	`int`, `double`, `bool`, `char`, `DateTime`	`string`, массивы, `List<T>`, `Random`

> Встроенные `int`, `double`, `bool` — это тоже структуры (`System.Int32`, `System.Double`, ...). Поэтому `Pixel` ведёт себя «как `int`», а `PixelClass` — «как массив».

Что лежит в переменной

Представим две переменные:

```
Pixel      a = new Pixel(10, 20, 30);
PixelClass c = new PixelClass(10, 20, 30);
```



```
                                        | R=10 | G=20 | B=30       |
                                        └──────────────────────────┘
```

Переменная `a` **и есть** пиксель. Переменная `c` — это **«ярлык»**, указывающий на пиксель, который лежит где-то в куче.

Хорошая аналогия:

- структура — это **листок бумаги с записанными данными**. Отдали кому-то ксерокопию — он может черкать в ней, ваш листок не изменится;
- класс — это **документ в Google Docs**. Отдали кому-то **ссылку** — он правит тот же самый документ, что и вы.

Где живут данные: стек и куча

- **Стек** — область памяти, где лежат локальные переменные и параметры текущего метода. Память выделяется и освобождается автоматически при входе в метод и выходе из него. Очень быстро.
- **Куча (heap)** — область памяти для объектов, которые создаются через `new` у **классов**. Объект живёт, пока на него есть хоть одна ссылка; когда ссылок не осталось, его со временем удаляет **сборщик мусора (GC)**.

Важное уточнение: фраза «структуры лежат в стеке» — упрощение. Правильно так: **структура хранится там, где объявлена**.

- локальная переменная `Pixel p` — в стеке метода;
- поле `Pixel` внутри объекта класса — внутри этого объекта, т.е. в куче;
- элементы массива `Pixel[]` — внутри массива, а массив (это класс!) — в куче.

Например, `Image` — класс, его поле `_pixels` — ссылка на массив `Pixel[,]` в куче, и **все пиксели лежат внутри этого массива одним сплошным блоком**.

Копирование: присваивание, методы, `ref`

Присваивание

```
Pixel a = new Pixel(10, 20, 30);
Pixel b = a;      // b — НОВАЯ независимая копия всех полей a
b.R = 99;         // a.R по-прежнему 10

PixelClass c = new PixelClass(10, 20, 30);
```

```
PixelClass d = c; // d — ещё одна ссылка на ТОТ ЖЕ объект
d.R = 99;         // c.R теперь тоже 99
```

Передача в метод

В C# параметры по умолчанию передаются **по значению** — метод получает копию того, что лежит в переменной:

- для структуры — копию **данных**. Изменения внутри метода снаружи не видны;
- для класса — копию **ссылки**. Через неё можно менять поля **того же** объекта — это видно снаружи. Но если внутри метода присвоить параметру **новый** объект (`p = new PixelClass(...)`), поменяется только локальная копия ссылки — переменная снаружи продолжит указывать на старый объект.

Это та же история, что и с массивами в лабораторной работе 3 (`BrokenSafeAppend` и `RefSafeAppend`).

`ref` — передача по ссылке

С `ref` метод получает не копию, а **саму переменную** вызывающего кода:

```
static void MakeWhiteRef(ref Pixel p) { p.R = 255; p.G = 255; p.B = 255; }

Pixel px = new Pixel(0, 0, 0);
MakeWhiteRef(ref px);   // px теперь (255, 255, 255)
```

Для структур `ref` даёт ещё и выигрыш в скорости, если структура большая: не нужно копировать все поля при каждом вызове.

`ref`-переменные и `ref`-возврат

Метод может вернуть не значение, а **ссылку на переменную** — например, на элемент массива:

```
// внутри класса Image
public ref Pixel GetPixel(int x, int y)
{
    return ref _pixels[x, y];
}
```

```
ref Pixel p = ref image.GetPixel(10, 20); // p — «псевдоним» пикселя внутри картинки
p.R = 255;                                // меняется пиксель В САМОМ изображении
```

Без `ref` слева (`Pixel p = image.GetPixel(10, 20);`) вы получите обычную копию.

Ловушки копирования (ошибки CS1612 и CS1654)

Свойство и индексатор возвращают копию

Индексатор `image[x, y]` и свойства — это на самом деле **методы** (`get`). Метод, возвращающий структуру, возвращает её **копию** — временное значение, которое нигде не сохранено. Поэтому:

```
image[0, 0].R = 255;   // ОШИБКА CS1612: Cannot modify the return value of
'Image.this[int, int]'
                       // because it is not a variable
```

Компилятор спасает вас: если бы это скомпилировалось, вы бы изменили временную копию, и она бы тут же исчезла — картинка не изменилась бы.

Правильные способы:

```
// Способ 1: копия → изменение → запись обратно
Pixel p = image[0, 0];
p.R = 255;
image[0, 0] = p;

// Способ 2: ссылка на пиксель
ref Pixel r = ref image.GetPixel(0, 0);
r.R = 255;
```

А вот **элемент массива** — это настоящая переменная (место в памяти), поэтому так можно:

```
Pixel[] pixels = new Pixel[3];
pixels[0].R = 255;     // OK: меняем поле прямо внутри массива
```

С `List<Pixel>` так нельзя (`list[0].R = 255` → CS1612): у списка индексатор — метод, а не элемент массива.

Переменная цикла `foreach`

```
foreach (Pixel p in pixels)
    p.R = 0;           // ОШИБКА CS1654: Cannot modify members of 'p'
                       // because it is a 'foreach iteration variable'
```

Переменная `p` — копия элемента, и менять её запрещено. Для `PixelClass[]` такая строка скомпилируется и сработает: `p` — копия **ссылки**, а объект тот же.

> Общий вывод: **изменяемые структуры** (с публичными полями, которые можно менять) — частый источник ошибок «изменил копию вместо оригинала». Поэтому в реальных проектах структуры часто делают неизменяемыми (`readonly struct`). Нам же изменяемый `Pixel` нужен как раз для того, чтобы увидеть эти эффекты.

Значения по умолчанию: `default` и `null`

- Переменная-структура **всегда** содержит данные. «Пустая» структура — это `default` : все поля обнулены. `new Pixel()` — то же самое, что `default(Pixel)` , т.е. `(0, 0, 0)` .
- Переменная-класс может не указывать ни на какой объект — тогда в ней `null` . Обращение к полю через `null` приводит к `NullReferenceException` (программа падает).

```
Pixel[] structs = new Pixel[3];        // 3 готовых пикселя (0, 0, 0)
structs[0].R = 5;                      // OK

PixelClass[] objects = new PixelClass[3]; // 3 ссылки, все null. Ни одного пикселя
ещё нет!
objects[0].R = 5;                      // NullReferenceException
objects[0] = new PixelClass(0, 0, 0);  // каждый объект нужно создать отдельно
```

Сколько это стоит в памяти

Посмотреть, сколько памяти выделила программа, можно методом `GC.GetAllocatedBytesForCurrentThread()` — он возвращает общее число байт, выделенных текущим потоком с начала работы. Замер «после − до» показывает, сколько выделила операция между ними.

Как устроены массивы из миллиона элементов (64-битная программа):

```
Pixel[] — один объект в куче:

┌─────────┬─────────┬─────────┬─────────┬─────┬─────────┐
|заголов.| R G B   | R G B   | R G B   | ... | R G B   |   ≈ 3 байта на пиксель
└─────────┴─────────┴─────────┴─────────┴─────┴─────────┘
```

```
PixelClass[] — массив ссылок + миллион отдельных объектов:

┌──────────┬──────┬──────┬──────┬─────┐
│заголов.  │ ref  │ ref  │ ref  │ ... │                    8 байт на ссылку
└──────────┴──┬───┴──┬───┴──┬───┴─────┘
              ▼      ▼      ▼
          [объект][объект][объект] ...                    ≈ 24 байта на объект
```

Почему объект класса с тремя `byte` занимает 24 байта? У **каждого** объекта в куче есть служебные данные: заголовок (8 байт) и указатель на описание типа (8 байт). Плюс 3 байта полей, а итог выравнивается до кратного 8. У структуры в массиве никаких служебных данных нет — только поля.

Поэтому для **большого количества маленьких значений** (пиксели, точки, координаты) структуры дают:

- в разы меньше памяти;
- один объект в куче вместо миллиона — меньше работы сборщику мусора;
- данные лежат подряд — процессору быстрее их читать (вспомните кэш из лабораторной работы 2).

Устройство типа: поля, конструкторы, свойства, методы

Всё, что ниже, одинаково пишется и в классе, и в структуре.

```
public struct Pixel
{
    // 1. Поля — данные
    public byte R, G, B;

    // 2. Конструктор — задаёт начальное состояние. Имя = имя типа, возвращаемого
типа нет.
    public Pixel(byte r, byte g, byte b)
    {
        R = r;
        G = g;
        B = b;
    }

    // 3. Вычисляемое свойство (только чтение) — выглядит как поле, но каждый раз
вычисляется
    public byte Brightness
    {
```

```
        get { return (byte)(0.3 * R + 0.59 * G + 0.11 * B); }
    }
    // короткая запись того же самого:
    // public byte Brightness => (byte)(0.3 * R + 0.59 * G + 0.11 * B);

    // 4. Метод экземпляра — работает с полями КОНКРЕТНОГО пикселя
    public override string ToString()
    {
        return $"({R}, {G}, {B})";
    }

    // 5. Статический метод — принадлежит ТИПУ, а не конкретному пикселю.
    //    Вызывается через имя типа: Pixel.Clamp(300)
    public static byte Clamp(int value)
    {
        if (value > 255) return 255;
        if (value < 0) return 0;
        return (byte)value;
    }
}
```

Экземплярный или статический? Если члену нужны данные конкретного объекта (`R`, `G`, `B` вот этого пикселя) — он экземплярный (`p.Brightness`). Если он работает только со своими параметрами — статический (`Pixel.Clamp(x)`, `Pixel.Gray(128)`). Класс, у которого **все** члены статические (как наш `Filters`), можно пометить `static class` — тогда создать его объект нельзя, он просто «папка для методов».

Про `override ToString()`. У любого типа в C# есть метод `ToString()`, который вызывается, например, в `Console.WriteLine(p)` и в `$"{p}"`. По умолчанию он печатает имя типа (`MyProject.Pixel`). Ключевое слово `override` означает «заменяю стандартную реализацию своей».

Особенности конструкторов у структур:

- у структуры **всегда** есть конструктор без параметров (`new Pixel()`), даже если вы написали свой с параметрами — он просто обнуляет все поля;
- у класса конструктор без параметров «исчезает», как только вы написали любой свой конструктор (`new PixelClass()` перестанет компилироваться).

Инициализатор объекта работает и с классами, и со структурами:

```
Pixel p = new Pixel { R = 10, G = 20, B = 30 };
```

Сравнение: `==` и `Equals`

Как это работает «из коробки»

```
PixelClass c1 = new PixelClass(1, 2, 3);
PixelClass c2 = new PixelClass(1, 2, 3);
PixelClass c3 = c1;

c1 == c2      // False — это РАЗНЫЕ объекты, хоть и с одинаковыми полями
c1.Equals(c2) // False — по умолчанию тоже сравнение ссылок
c1 == c3      // True  — это один и тот же объект

Pixel p1 = new Pixel(1, 2, 3);
Pixel p2 = new Pixel(1, 2, 3);

p1.Equals(p2) // True — у структур Equals по умолчанию сравнивает поля
p1 == p2      // ОШИБКА CS0019: Operator '==' cannot be applied to operands of type
'Pixel' and 'Pixel'
```

То есть у классов «равны» по умолчанию означает «**это тот же самый объект**» (идентичность), а у структур — «**у них одинаковое содержимое**» (равенство значений). Оператора `==` для своих структур компилятор не создаёт — его нужно написать самому.

Как добавить `==` в структуру

```
public struct Pixel
{
    // ... поля, конструктор ...

    public static bool operator ==(Pixel a, Pixel b)
    {
        return a.R == b.R && a.G == b.G && a.B == b.B;
    }

    public static bool operator !=(Pixel a, Pixel b)
    {
        return !(a == b);
    }

    // Equals и GetHashCode нужно переопределить вместе с ==,
    // чтобы все способы сравнения давали одинаковый ответ (иначе компилятор выдаст
```

```
предупреждения).
    public override bool Equals(object obj)
    {
        return obj is Pixel other && this == other;
    }

    public override int GetHashCode()
    {
        return HashCode.Combine(R, G, B);
    }
}
```

- Операторы всегда `public static`, и `==` с `!=` объявляются **парой**.
- `GetHashCode()` используется словарями (`Dictionary`) и множествами (`HashSet`): равные объекты **обязаны** иметь одинаковый хэш-код. `HashCode.Combine` собирает хэш из нескольких полей.

Мелкая и глубокая копия

Иногда копия **нужна** (например, для размытия). У любого массива есть метод `Clone()`, который создаёт новый массив и копирует в него **элементы**. Результат имеет тип `object`, его нужно привести:

```
Pixel[,] copy = (Pixel[,])_pixels.Clone();
```

Что именно скопируется, зависит от типа элементов:

- `Pixel[,]` — элементы это сами данные пикселей ⇒ получаем **полностью независимую** копию (глубокую);
- `PixelClass[,]` — элементы это ссылки ⇒ скопируются **ссылки**, а объекты-пиксели останутся общими для обоих массивов (**мелкая копия**). Изменили пиксель через копию — он изменился и в оригинале.

Чтобы сделать глубокую копию массива объектов, нужно вручную создать новый объект для каждого элемента.

Когда выбирать структуру, а когда класс

Структура, если одновременно:

- тип описывает одно **значение** (цвет, точку, дату, комплексное число), а не «сущность» с собственной жизнью;

- он **маленький** (ориентир Microsoft — до 16 байт);
- таких значений будет **много** (массивы на миллионы элементов);
- копирование при присваивании — это нормальное, ожидаемое поведение.

Класс во всех остальных случаях, особенно если:

- объект большой или содержит коллекции (`Image` с миллионами пикселей);
- у объекта есть «личность»: изменения должны быть видны всем, кто на него ссылается (счёт в банке, игрок, окно программы);
- нужно наследование (будет позже в курсе) или `null` как «объекта нет».

Наш выбор в работе: `Pixel` — 3 байта, их миллионы, это просто «значение цвета» ⇒ **структура**. `Image` — большой объект, который передают в фильтры, чтобы они меняли **его** ⇒ **класс**.

> **Осторожно со структурой, внутри которой есть ссылка.** Если бы `Image` был структурой, при копировании скопировалось бы поле `_pixels` — а это **ссылка** на массив. В итоге две «копии» картинки делили бы один и тот же массив пикселей. Значимый тип копирует только то, что лежит непосредственно в его полях.

Класс `Image` («чёрный ящик»)

Код загрузки и сохранения BMP вы не пишете — файл `Image.cs` есть в курсе ЭИОС. Упрощённо он выглядит так:

```
public class Image
{
    private Pixel[,] _pixels;

    public int Width  => _pixels.GetLength(0);
    public int Height => _pixels.GetLength(1);

    public Image(string filePath) { /* ... загрузка BMP ... */ }
    public void Save(string filePath) { /* ... сохранение BMP ... */ }

    // Индексатор: get возвращает КОПИЮ пикселя, set записывает пиксель целиком
    public Pixel this[int x, int y]
    {
        get { return _pixels[x, y]; }
        set { _pixels[x, y] = value; }
    }
```

```
    // Ссылка на пиксель внутри массива
    public ref Pixel GetPixel(int x, int y)
    {
        return ref _pixels[x, y];
    }
}
```

Обратите внимание: поле `_pixels` — `private`. Снаружи нельзя добраться до массива напрямую, только через индексатор и `GetPixel`. Это **инкапсуляция**: класс сам решает, как именно к его данным можно обращаться.

Проход по всем пикселям (базовый шаблон)

```
public static void ApplyFilter(Image image) // поменяйте название на название вашего фильтра
{
    for (int y = 0; y < image.Height; y++)      // по высоте (строки)
    {
        for (int x = 0; x < image.Width; x++)   // по ширине (столбцы)
        {
            Pixel p = image[x, y];   // получили КОПИЮ пикселя

            // ... меняем p.R, p.G, p.B ...

            image[x, y] = p;         // записали копию обратно — без этой строки ничего не изменится!
        }
    }
}
```

Тот же проход через ссылку:

```
for (int y = 0; y < image.Height; y++)
{
    for (int x = 0; x < image.Width; x++)
    {
        ref Pixel p = ref image.GetPixel(x, y);   // ссылка на пиксель в изображении
        // ... меняем p.R, p.G, p.B — изменения сразу попадают в картинку ...
    }
}
```

Ограничение значений (Clamping)

Канал цвета не может быть меньше 0 или больше 255. Если вычисления вышли за эти рамки, их нужно «обрезать». Считать удобно в `int`, а в `byte` переводить в самом конце:

```
Pixel p = image[x, y];
int newR = p.R + 50;        // byte + int = int, переполнения нет
if (newR > 255) newR = 255;
if (newR < 0) newR = 0;
p.R = (byte)newR;
image[x, y] = p;
```

В этой работе эта логика оформляется один раз — в методе `Pixel.Clamp`.

Про byte и sbyte здесь.

В C# есть два типа данных, чтобы хранить целое число в одном байте (8 бит). Отличие между этими типами данных - **ЗНАК**.

1. byte (Беззнаковый, Положительный)

- **Что это такое?**
  8-битное **беззнаковое** целое число. Ключевое слово - "беззнаковое".
- **Диапазон: 0 до 255**
  Всего 256 возможных значений.
- **Почему так?**
  У него нет бита, отвечающего за знак "минус". Все 8 бит работают на то, чтобы показать величину числа. Данный тип данных не знает про отрицательные числа. Его одометр может идти только вперед, от 0 до 255.
- **Где может использоваться? (99% случаев)**
  - **ЦВЕТА (R, G, B, A):** Наш главный пример. Яркость цветового канала не может быть отрицательной. Она либо есть (0-255), либо ее нет. Поэтому `Pixel { public byte R, G, B; }` — это *единственно верный путь*.
  - **РАБОТА С ФАЙЛАМИ/СЕТЬЮ:** Когда мы читаем файл или получаем данные из сети это будет поток **сырых байтов**. Это просто данные, у них нет знака.
  - Любые данные, где отрицательное значение не имеет смысла: количество патронов, процент здоровья, номер символа в кодировке ASCII.

Идеален, когда нам нужно число от 0 до 255. Сомнения? Берем `byte`.

---

2. sbyte (Знаковый)

- **Что это такое?**
  8-битное **знаковое** целое число. s~~(как доллар)~~ в начале — это signed, "знаковый".
- **Диапазон: -128 до 127**
  Всего те же 256 возможных значений, но диапазон "сдвинут" в отрицательную область.
- **Почему так?**
  Этот тип данных тратит один из своих восьми бит (старший бит) на то, чтобы хранить запись, положительный он или отрицательный. Из-за этого на саму величину числа у него остается только 7 бит.
  (А то, что он уходит до -128, а не -127 — это "магия" дополнительного кода).
- **Где используется?**
  - **Изменения/Дельты:** "изменение температуры на -5 градусов", "смещение координат джойстика на +10 по оси X".
  - **Игровые параметры:** "бонус к силе -2", "модификатор урона +5".
  - Взаимодействие с другими языками (Java, например), где byte по умолчанию знаковый. Узкоспециализированный инструмент.

Побитовые операции

Число `byte` — это 8 бит. Побитовые операторы работают с каждым битом по отдельности:

Оператор	Название	Пример (`a = 0b1100_1010`)
`~a`	НЕ (инверсия всех битов)	`0b0011_0101`
`a & b`	И (бит = 1, если в обоих 1)	`a & 0b0000_1111` = `0b0000_1010`
`a | b`	ИЛИ (бит = 1, если хоть в одном 1)	`a | 0b0000_0001` = `0b1100_1011`
`a << n`, `a >> n`	сдвиг влево / вправо на n бит	`a >> 4` = `0b0000_1100`

Инверсия и негатив. Инвертировать все 8 бит байта — то же самое, что вычесть его из 255 (`1111_1111`): каждая единица становится нулём и наоборот. Например, `200 = 1100_1000`, `~200 → 0011_0111 = 55 = 255 − 200`.

Но в C# побитовые операции над `byte` выполняются в типе `int` (32 бита): `~p.R` инвертирует **все 32 бита**, и получается отрицательное число. Приведение `(byte)` отбрасывает старшие 24 бита и оставляет нужные 8:

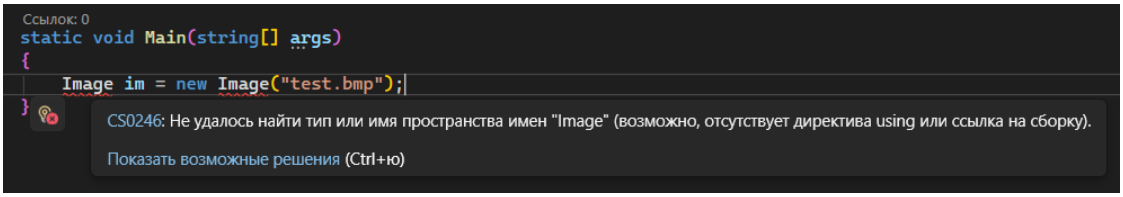
```
byte r = 200;
int  wide = ~r;          // -201 (лишние единицы в старших битах)
byte inv  = (byte)~r;    // 55
```

Маски. Чтобы прочитать или изменить отдельные биты, используют «маски»:

```
int lowBit = value & 1;                          // прочитать младший бит
value = (byte)(value & 0b1111_1110);             // обнулить младший бит
value = (byte)(value | 1);                       // установить младший бит в 1
value = (byte)((value & 0b1111_1110) | bit);     // записать в младший бит значение
bit (0 или 1)
```

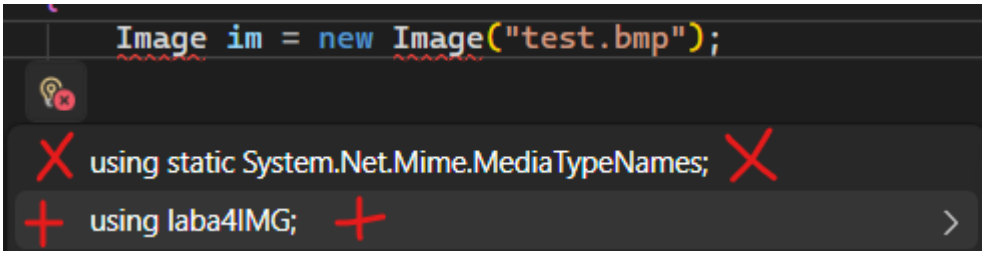
Подключение пространства имен

При создании объекта класса `Image` вы можете столкнуться с подобной ошибкой:

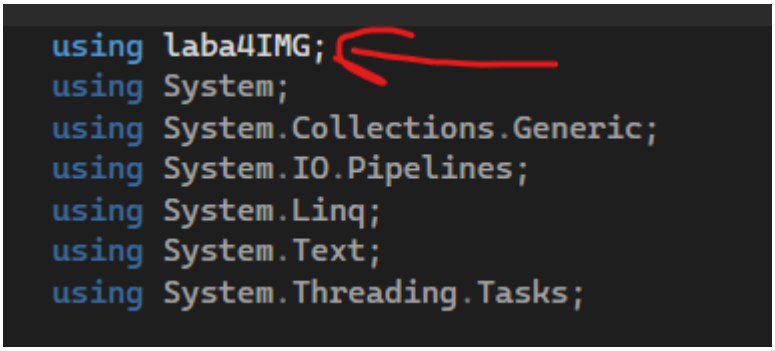


Наша программа не может найти созданный нами тип данных `Image`, нам нужно явно указать пространство имен в котором находится этот класс.

При нажатию на "лампочку с крестиком" вы увидите 2 вариант:

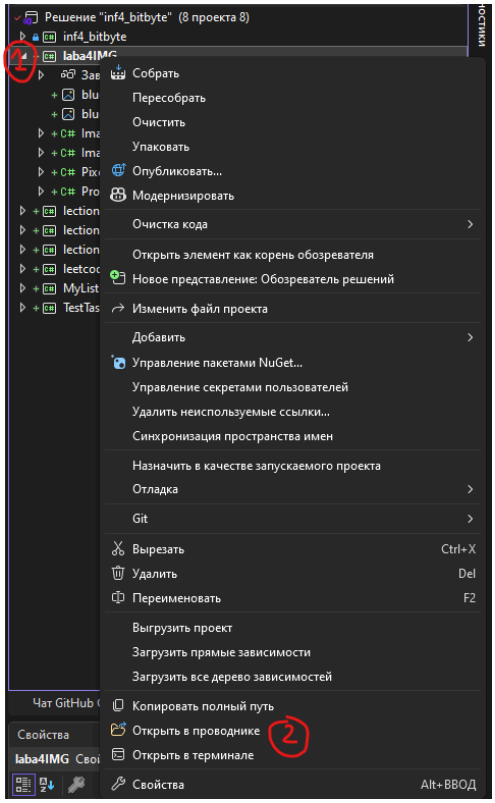


Нужно выбрать вариант с названием вашего проекта. После этого в файле `Program.cs` в самом верху появится строка:



Как открыть папку с программой

Для открытия папки с проектом нажмите ПКМ по **ПРОЕКТУ** (зелененькая иконка) ⇒ Открыть в проводнике:



Далее по примерно такому пути: `laba4IMG\bin\Debug\net9.0` (вместо `net9.0` может быть `net8.0` — зависит от версии .NET в проекте) будут находиться выходные файлы (exe для запуска).

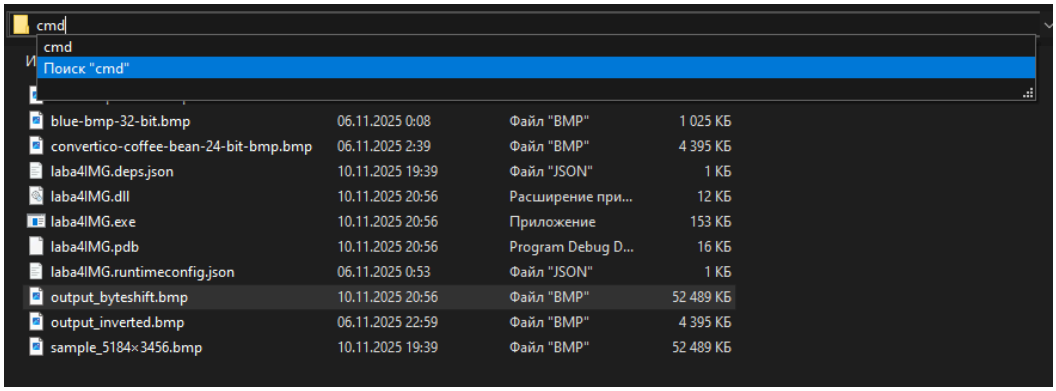
Именно в этой папке должны находиться картинки, именно сюда они будут сохраняться.

Считывание из аргументов командной строки (cmd)

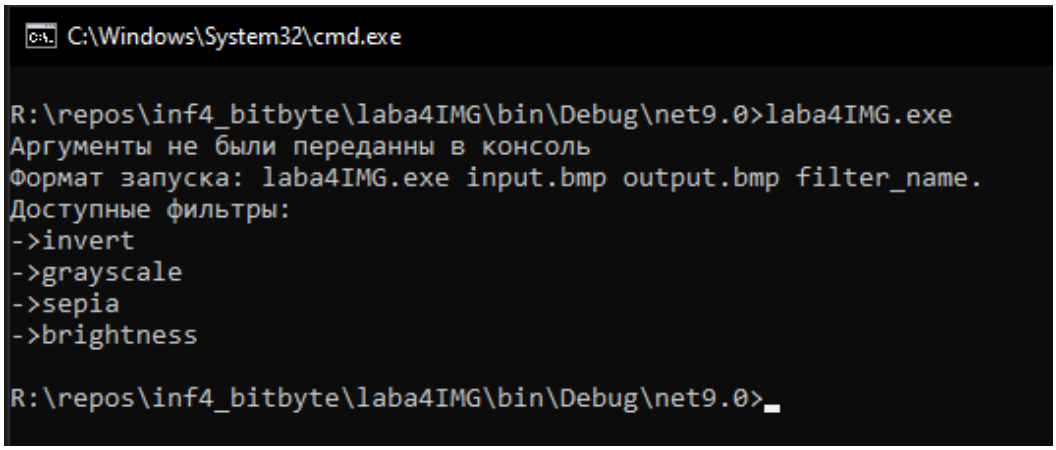
В папке `bin` вашего проекта вы увидите тот самый заветный `file.exe` файл, который и является вашей программой. Вы можете кликнуть по нему 2 раза и он стандартно запустится. Если у вас нет никакой обработки после выполнения программы - программа выполнится и закроется после выполнения.

Чтобы увидеть результат мы можем запустить программу из консоли:

- Если ввести `cmd` в проводнике - откроется консоль.



- Вводим название `file.exe`



- Программа закрылась, но мы видим результат.

Мы можем передать дополнительные аргументы в нашу программу. В коде за это отвечает аргумент `string[] args` в методе `Main`. Данный аргумент является массивом строк, при запуске программы аргументы, введённые через пробел (`proga.exe input.bmp output.bmp invert`), автоматически складываются в этот массив. Соответственно:

- `input.bmp` будет записан в `args[0]`
- `output.bmp` будет записан в `args[1]`
- `invert` будет записан в `args[2]`

Таким образом мы получаем еще один способ ввода данных в нашу программу на этапе выполнения.

> Класс `Image` умеет работать только с **24-битными BMP без сжатия**. JPG и PNG он не откроет — сначала пересохраните картинку в BMP (например, в Paint: «Сохранить как» → «24-разрядный рисунок»).

Важно: все эти аргументы приходят в виде строки. Если вы хотите ввести и использовать число - его нужно сконвертировать в число.

```
static void Main(string[] args)
{
    // Блок кода срабатывает, если мы передали 0 аргументов.
    // Пример такого вызова программы выше на скрине.
    if (args.Length == 0)
    {
        Console.WriteLine("Аргументы не были переданы в консоль");
        Console.WriteLine("Формат запуска: laba4IMG.exe input.bmp output.bmp
filter_name");
        Console.WriteLine("Доступные фильтры:");
```

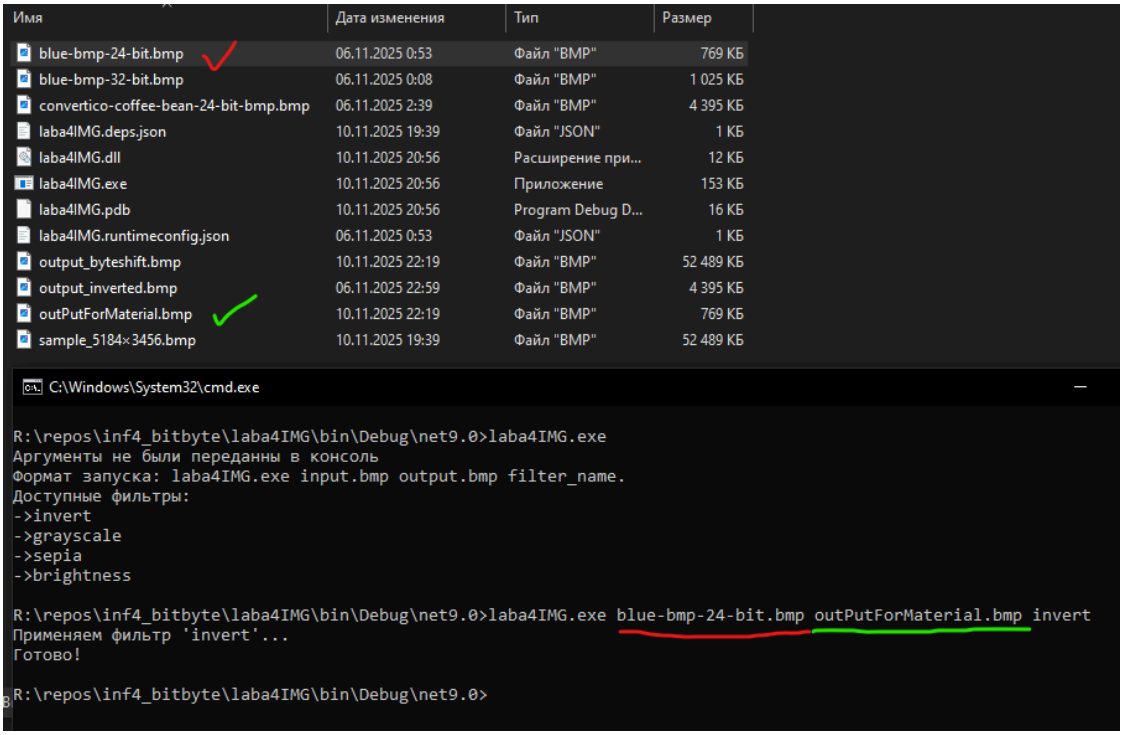
```
        Console.WriteLine("->invert");
        Console.WriteLine("->grayscale");
        Console.WriteLine("->brightness <число>");
        Console.WriteLine("->blur");

        return;
    }

    // Если передано меньше минимально нужного количества аргументов.
    if (args.Length < 3)
    {
        Console.WriteLine("Вы передали мало аргументов");
        return; // без return дальше будет обращение к args[2] и падение программы
    }

    // Парсим переданные аргументы в путь до картинки, куда сохранять и какой фильтр
выбран.
    string inputPath = args[0];
    string outputPath = args[1];
    string filterName = args[2].ToLower();
}
```

Пример выполнения программы через консоль:

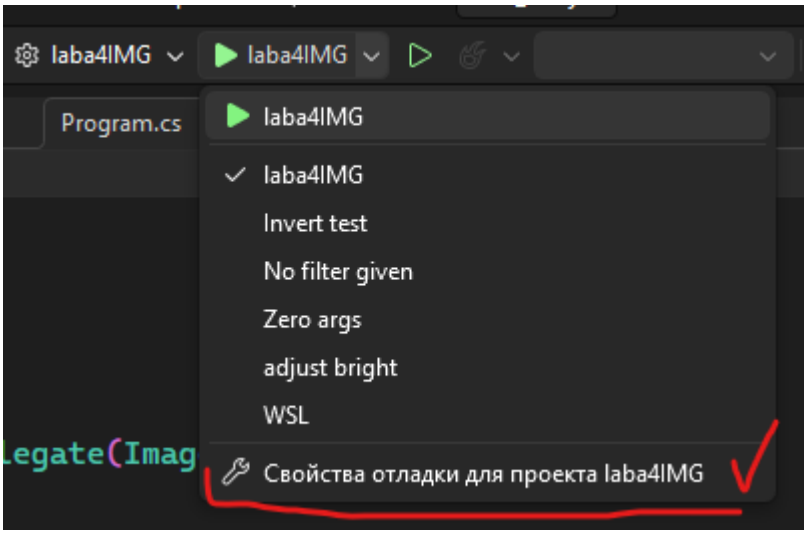


Добавление профилей отладки консоли

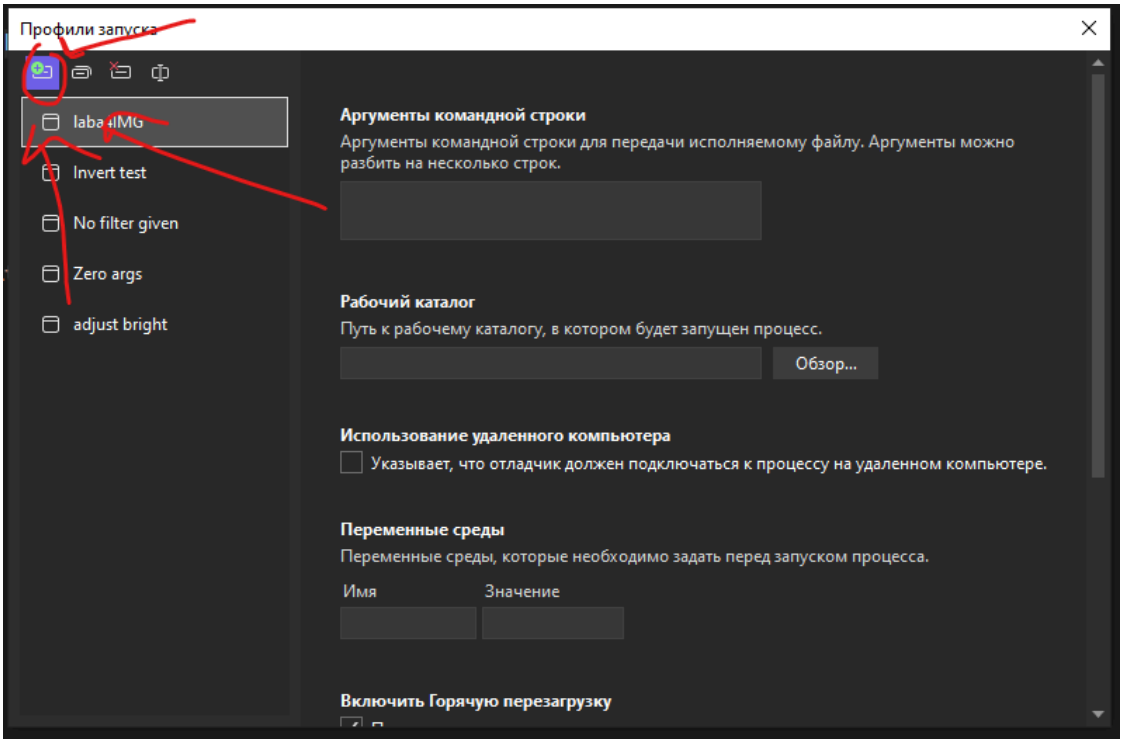
Может возникнуть ситуация: мы отлаживаем нашу программу и передачу аргументов из консоли. В таком случае мы не сможем отлаживать и смотреть значения переменных и где ломается программа в `Visual Studio`.

Для этого мы можем создать "профиль отладки" и запускать нашу программу через `Visual Studio` и с переданными в консоль аргументами. Для этого:

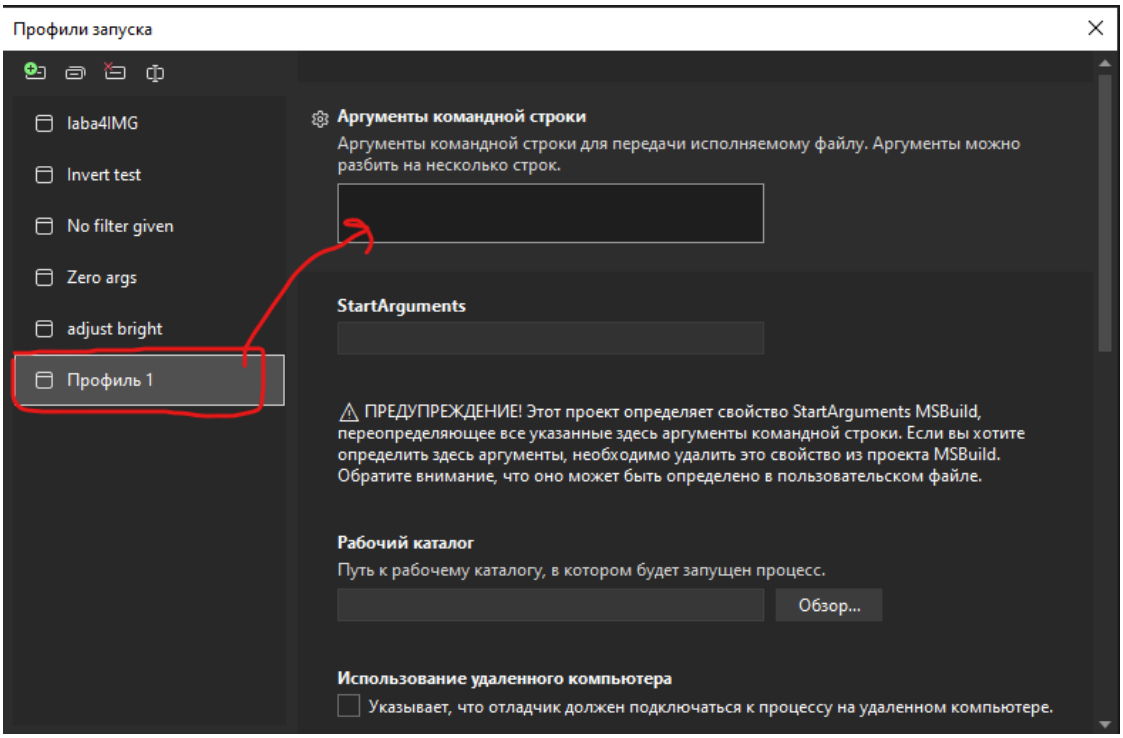
1. Нужно задать на стрелочку вниз рядом с кнопкой запуска проекта. Выбрать "свойства отладки для проекта `laba4IMG`".



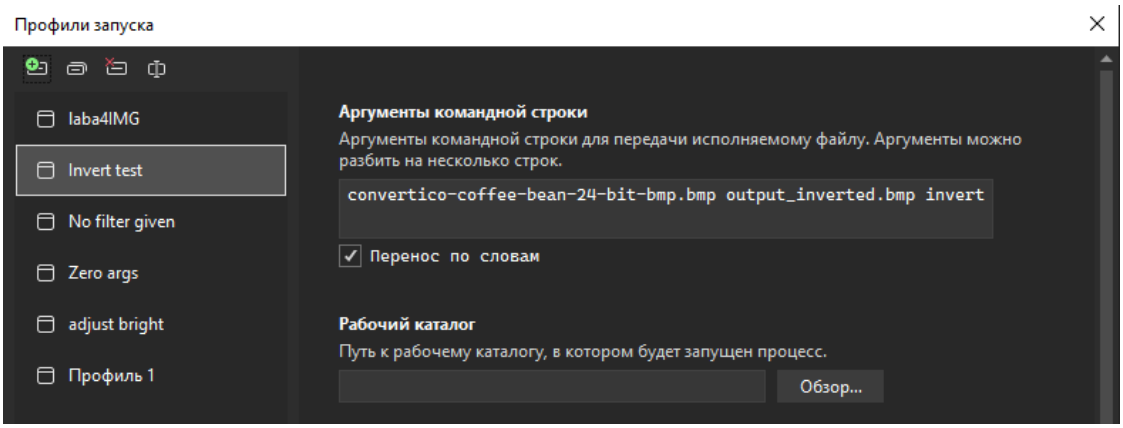
Теперь нужно `создать новый профиль` → `Проект`:



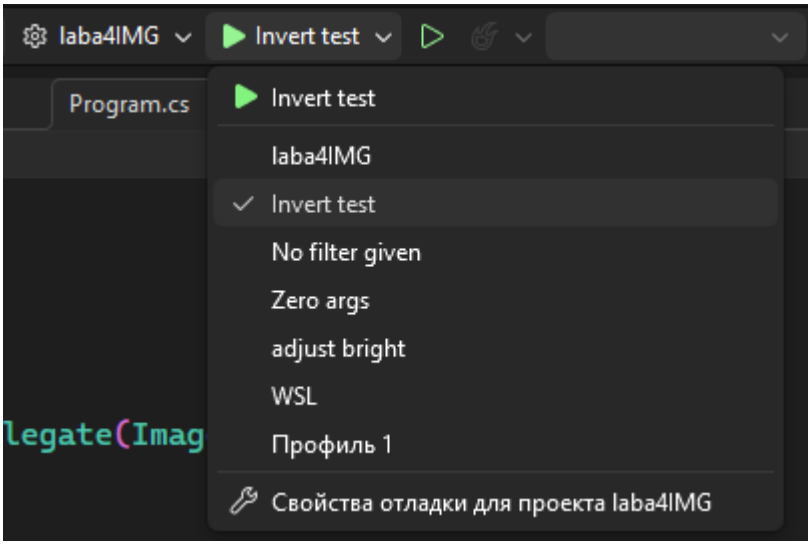
Теперь мы можем изменить название этого профиля и вписать аргументы, которые передадутся в консоль при запуске.



Пример переданных аргументов:



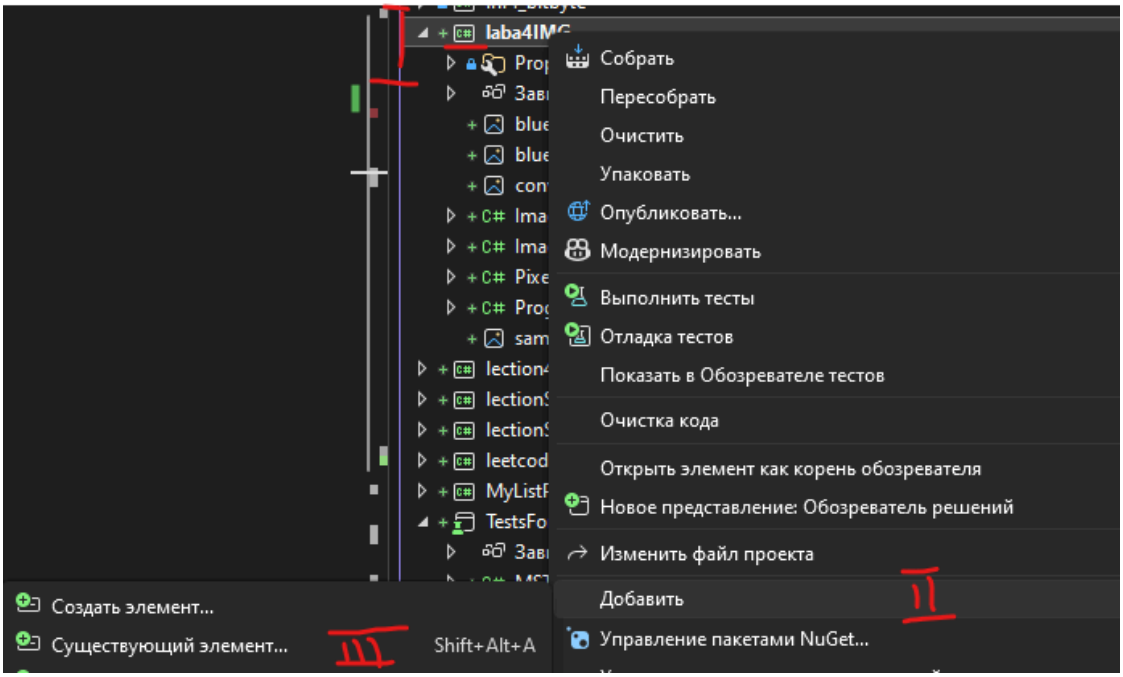
Теперь мы можем отладить нашу программу, передавая аргументы в консоль во время отладки.



Добавление классов в проект (создание/импорт)

Для добавление **существующего** файла в проект нужно:

- Кликнуть ПКМ по названию проекта (с зеленой иконкой в обозревателе решений слева)
- Кликнуть по "Добавить"
- Выбрать "Существующий элемент" Далее просто выбираем файл (`Image.cs` или `Pixel.cs`). После в нашем проекте внутри **обозревателя решений** отобразится файл.



Внутри этого файла нужно поменять пространство имен на ваше.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

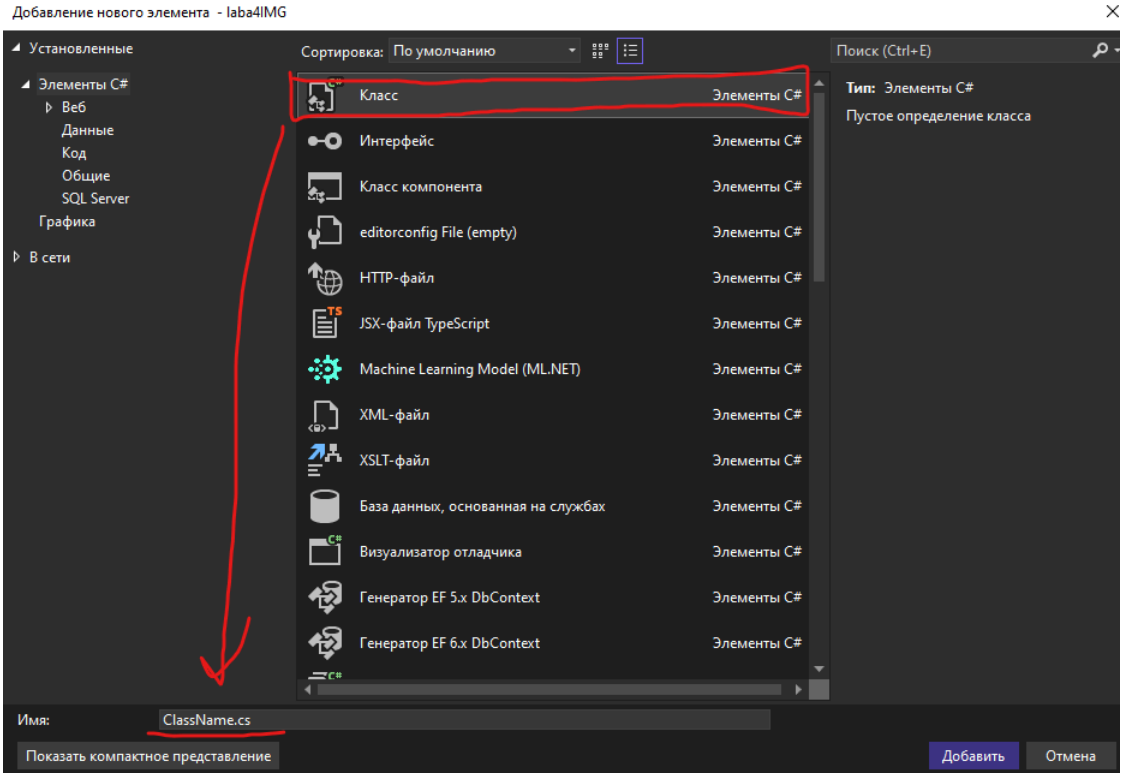
namespace laba4IMG
{
    /// <summary>
    /// Класс для загрузки, сохранения и изменения изображений в формате BMP (24-бит).
    /// </summary>
    Ссылок: 12
    public class Image
    {
        // Приватное поле, где реально хранятся все пиксели.
        // Это и есть наше изображение в виде двумерного массива.
        private Pixel[,] _pixels;
```

Обведенную строку нужно заменить на название вашего проекта.

Для создания нового файла путь примерно такой же:

- Кликнуть ПКМ по названию проекта (с зеленой иконкой в обозревателе решений слева)
- Кликнуть по "Добавить"
- Выбрать "Создать элемент..."

Выбираем "Класс", даем ему название. Класс создастся с таким же названием, как файл.



Для создания структуры путь такой же, только нужно в созданном классе поменять `class` на `struct`.

Пространства имен

Пространство имен (namespace) — еще один способ организации кода, который заключается в возможности классы, типы и другие пространства имен объеденять в однин "контейнер".

Простая аналогия: Думайте о пространствах имен как о папках на вашем компьютере. В папке "Фотографии" вы храните изображения, а в папке "Документы" — текстовые файлы. Точно так же в пространстве имен `MyProject.UI` могут храниться классы, отвечающие за пользовательский интерфейс, а в `MyProject.Data` — классы для работы с данными.

Зачем нужны пространства имен?

У пространств имен есть две основные цели:

1. **Организация кода:** Они помогают структурировать большие проекты, делая код более понятным и читаемым. Вы всегда знаете, где искать нужный класс.
2. **Предотвращение конфликтов имен:** В большом проекте или при использовании сторонних библиотек могут появиться классы с одинаковыми именами. Пространства имен решают эту проблему. Например, у вас может быть класс `Logger` в вашем проекте, и библиотека, которую вы используете, тоже может иметь класс `Logger`. Поместив их в разные пространства имен (например, `MyProject.Logging` и `ThirdPartyLibrary.Logging`), вы избежите путаницы.

Как объявить и использовать пространство имен?

Объявление

Для создания пространства имен используется ключевое слово `namespace`, за которым следует его имя.

```
namespace MyProject
{
    // Здесь находятся ваши классы, структуры и т.д.
    class MyClass
    {
        // ...
    }
}
```

Пространства имен могут быть вложенными, образуя иерархию, подобную папкам.

```
namespace MyProject.Data
{
    class DatabaseReader
    {
        // ...
    }
}
```

Не обязательно пространство имен должно называться как папка или проект в котором оно находится.

Использование

Есть два способа получить доступ к классу внутри пространства имен:

1. **Полное (уточненное) имя:** Указать всю цепочку пространств имен до имени класса через точку.

   ```
   MyProject.Data.DatabaseReader reader = new MyProject.Data.DatabaseReader();
   ```

2. **Директива `using`:** Чтобы не писать каждый раз длинное имя, можно "подключить" пространство имен в начале файла с помощью директивы `using`. Это говорит компилятору, где искать классы, которые вы используете.

   ```
   using System;
   using MyProject.Data; // Подключаем наше пространство имен

   class Program
   {
       static void Main(string[] args)
       {
           // Теперь можно использовать короткое имя
           DatabaseReader reader = new DatabaseReader();
           Console.WriteLine("Доступ к классу получен!");
       }
   }
   ```

Стандартные пространства имен .NET

Когда вы создаете проект, вы уже используете множество пространств имен из стандартной библиотеки классов .NET. Вот несколько ключевых примеров:

- `System`: Содержит фундаментальные и базовые классы, такие как `Console`, `String`, `Int32`.
- `System.IO`: Предоставляет классы для работы с файлами и потоками данных (например, `File`, `StreamReader`).
- `System.Collections.Generic`: Содержит классы для работы с коллекциями, такие как `List<T>` и `Dictionary<TKey, TValue>`.

Начиная с .NET 6 и C# 10, многие из этих часто используемых пространств имен подключаются к вашему проекту автоматически (неявно), поэтому вам не всегда нужно писать для них директиву `using`.